# Real-time with WebSockets



## Why it matters

Plain HTTP has one shape: the **client asks, the server answers, the exchange ends**. That request/response model is perfect for loading a page or saving a form, but it cannot *push*. If a message arrives on the server — a new chat line, a price tick, another user's cursor move — HTTP has no way to hand it to the browser until the browser asks again. For a whole class of products (chat, multiplayer games, live dashboards, collaborative editors, trading screens, ride tracking) that "ask again" model is either too slow or too wasteful to ship.

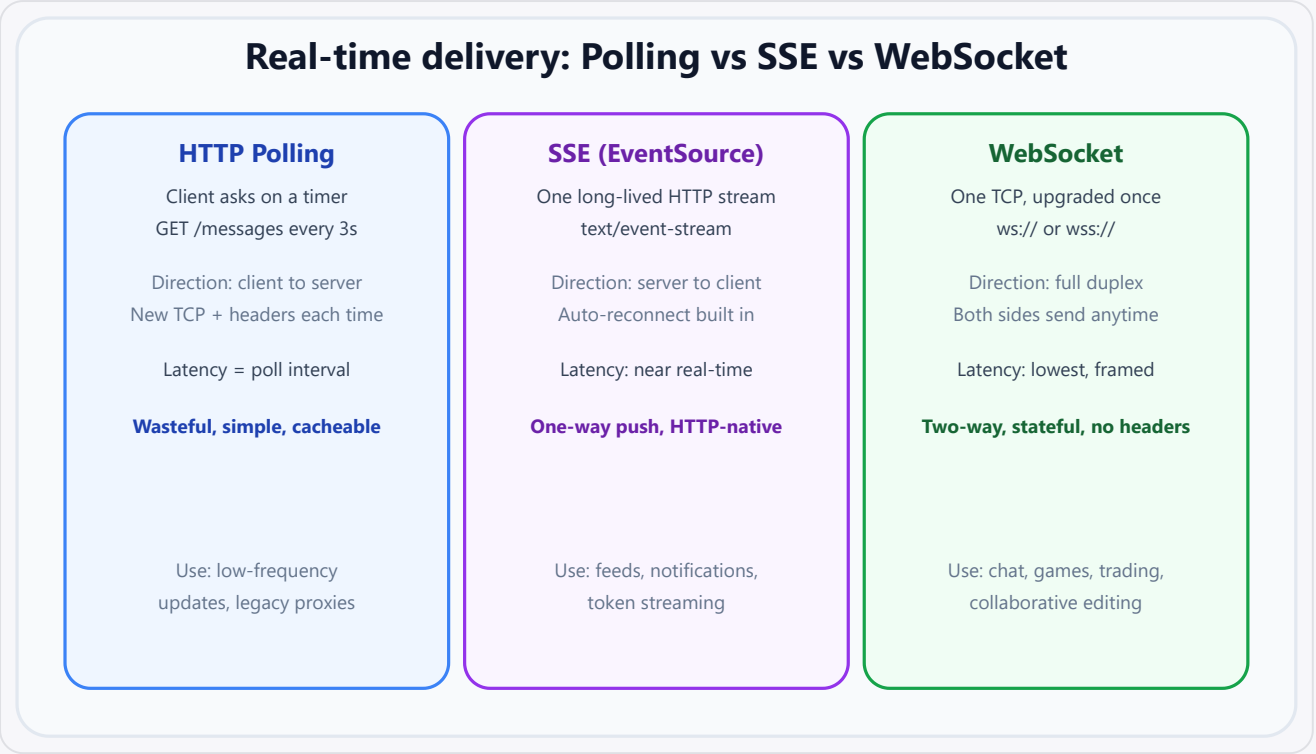
WebSockets close that gap. They give you a single, long-lived, **full-duplex** connection over which either side can send a message at any moment, with almost no per-message overhead. Knowing when to reach for WebSockets — and, just as important, when a cheaper tool like Server-Sent Events or ordinary polling is the correct answer — is a signature backend skill and a very common interview theme. Getting it wrong means either a laggy product or a server melting under connections it never needed to hold open.

## The concept, simply

A WebSocket is a persistent two-way pipe between client and server. It *starts* its life as a normal HTTP request, then asks to be "upgraded" to the WebSocket protocol. Once upgraded, the HTTP framing is gone: both ends now speak in lightweight **frames** over the same underlying TCP connection, and both can speak whenever they like without waiting to be asked.

Compare the three common real-time strategies. **Polling** is the browser repeatedly asking "anything new?" on a timer — simple, but latency equals your interval and most

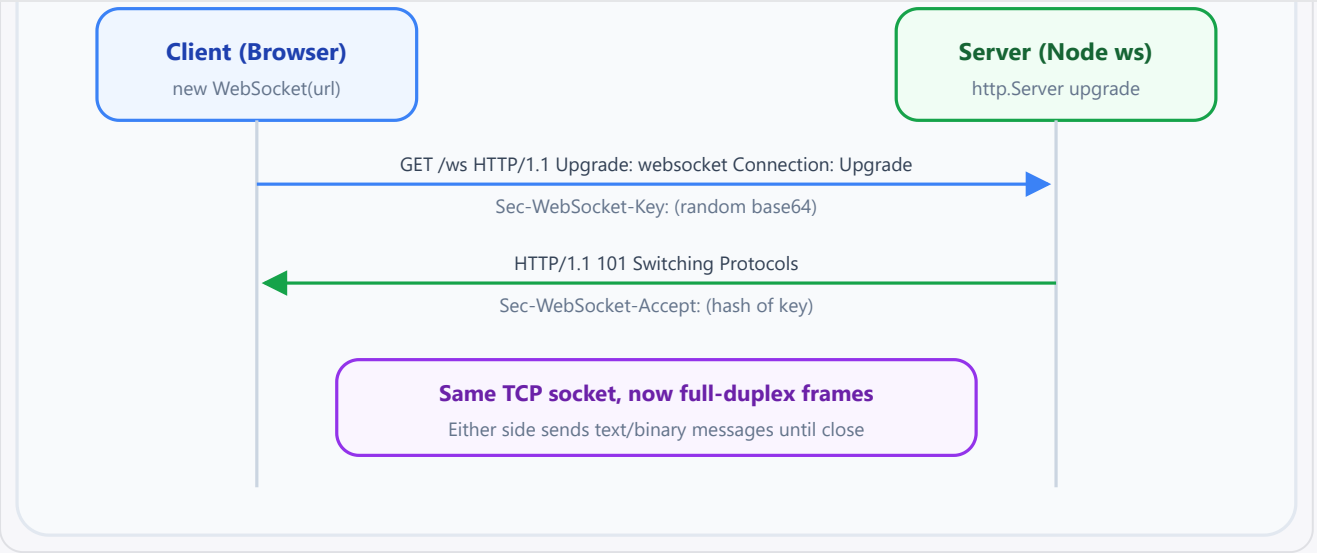
reconnecting, and ideal for feeds and notification streams. **WebSockets** are the heavyweight: genuinely two-way, lowest latency, but a stateful connection you now have to manage and scale.




Polling asks on a timer, SSE streams one-way over HTTP, and WebSocket is a full-duplex pipe — pick the lightest tool that meets the latency and directionality you actually need.

## How it works (deep)

The connection begins with a normal HTTP/1.1 GET carrying three tell-tale headers: `Upgrade: websocket`, `Connection: Upgrade`, and a random `Sec-WebSocket-Key`. If the server agrees, it replies with status **101 Switching Protocols** and a `Sec-WebSocket-Accept` header derived from that key (the client verifies it to prove it really reached a WebSocket server, not a confused cache). After the 101, the socket is no longer HTTP — the same TCP connection now carries WebSocket frames.




The handshake is one HTTP request that upgrades in place — after the 101, both sides send framed messages over the same TCP connection until either closes it.

Each message is sent as one or more frames. A frame header encodes an **opcode** (text, binary, ping, pong, or close), a payload length, and — for messages sent *from the browser* — a masking key XORed over the payload (a security measure against cache-poisoning proxies; servers never mask). Because framing is so thin (as few as 2 bytes of overhead), a WebSocket can push thousands of tiny updates far more cheaply than the same number of HTTP requests, each of which would carry full headers and cookies.

In Node, the `http.Server` emits an `'upgrade'` event instead of the usual `'request'` event when those upgrade headers arrive. The `ws` library hooks that event, completes the handshake, and hands you a socket object with `send()` and `'message'`/`'close'` events. `socket.io` sits a layer above: it adds rooms, automatic reconnection, acknowledgements, and a *fallback to HTTP long-polling* when a raw WebSocket cannot be established (hostile proxies, some corporate networks) — at the cost of its own wire protocol, so a socket.io client cannot talk to a bare `ws` server and vice-versa.

Two operational realities shape every production deployment. First, WebSockets are **stateful**: connection C lives on exactly one server instance. If user A is on instance 1 and user B is on instance 2, instance 1 cannot deliver a message to B on its own — you need a **backplane** (commonly a Redis pub/sub adapter) so instances can relay events to each other. Second, load balancers and reverse proxies must be told to allow the upgrade and

## Code examples

A minimal broadcast server with `ws`. Note the ping/pong heartbeat that detects dead peers — TCP will happily believe a yanked-cable connection is still "open" for minutes.

JAVASCRIPT Copy

```
import { WebSocketServer } from 'ws';

const wss = new WebSocketServer({ port: 8080 });

function heartbeat() { this.isAlive = true; }

wss.on('connection', (ws, req) => {
  ws.isAlive = true;
  ws.on('pong', heartbeat); // client answered our ping

  ws.on('message', (data, isBinary) => {
    // Broadcast to everyone else who is still open
    for (const client of wss.clients) {
      if (client !== ws && client.readyState === client.OPEN) {
        client.send(data, { binary: isBinary });
      }
    }
  });

  ws.on('close', () => console.log('client left'));
});

// Every 30s, drop peers that never ponged back
const interval = setInterval(() => {
  for (const ws of wss.clients) {
    if (ws.isAlive === false) { ws.terminate(); continue; }
    ws.isAlive = false;
    ws.ping();
  }
}, 30_000);
```

Sharing the same Node HTTP server between Express and WebSockets — you almost never want two ports. You authenticate *during* the upgrade so unauthenticated sockets never open.

JAVASCRIPT Copy

```
import http from 'node:http';
import express from 'express';
import { WebSocketServer } from 'ws';

const app = express();
app.get('/health', (_req, res) => res.send('ok'));

const server = http.createServer(app);
const wss = new WebSocketServer({ noServer: true }); // we drive the hands

server.on('upgrade', (req, socket, head) => {
  const token = new URL(req.url, 'http://x').searchParams.get('token');
  if (!isValid(token)) {
    socket.write('HTTP/1.1 401 Unauthorized\r
\r
');
    socket.destroy();
    return;
  }
  wss.handleUpgrade(req, socket, head, (ws) => {
    wss.emit('connection', ws, req);
  });
});

server.listen(3000);
function isValid(t) { return t === 'secret'; }
```

A socket.io server showing rooms — the feature you would otherwise hand-roll. Rooms let you address a subset of clients (one chat channel, one document) without tracking arrays yourself.

```
import { Server } from 'socket.io';

const io = new Server(3000, { cors: { origin: '*' } });

io.on('connection', (socket) => {
  socket.on('join', (roomId) => {
    socket.join(roomId);
    io.to(roomId).emit('system', socket.id + ' joined');
  });

  socket.on('chat', ({ roomId, text }) => {
    // Emit only to sockets in this room, excluding the sender
    socket.to(roomId).emit('chat', { from: socket.id, text });
  });
});
```

Scaling socket.io across instances with the Redis adapter — the backplane in one line, so an emit on instance 1 reaches a client held by instance 2.

JAVASCRIPT Copy

```
import { Server } from 'socket.io';
import { createAdapter } from '@socket.io/redis-adapter';
import { createClient } from 'redis';

const pub = createClient({ url: process.env.REDIS_URL });
const sub = pub.duplicate();
await Promise.all([pub.connect(), sub.connect()]);

const io = new Server(3000);
io.adapter(createAdapter(pub, sub)); // cross-instance fan-out

io.on('connection', (s) => s.on('broadcast', (m) => io.emit('broadcast', m
```

## Common mistakes & gotchas

- **Assuming a connection is alive because TCP says so.** A crashed client or a dropped Wi-Fi connection can leave a "half-open" socket that never fires `'close'`. Without a ping/pong heartbeat you accumulate zombie connections and leak memory.
- **Forgetting the multi-instance problem.** Everything works on your laptop with one process. In production behind a load balancer, a naive `broadcast to all clients` only reaches the clients on *that* instance. You need a Redis (or similar) backplane.
- **Not enabling sticky sessions** when using socket.io's HTTP long-polling fallback. The initial handshake and subsequent polls must hit the same instance, or the session breaks. Raw WebSockets avoid this because there is a single upgraded connection.
- **Trusting the `Origin` header is set — but never checking it.** Browsers do not enforce the same-origin policy on WebSockets. Any website can open a socket to your server ("Cross-Site WebSocket Hijacking"). Validate `Origin` and authenticate on upgrade.
- **Sending giant JSON blobs as text.** Frames are cheap but your JSON parsing is not. High-frequency streams (game state, market data) usually belong in a compact binary format, not `JSON.stringify` on every tick.
- **No backpressure handling.** If you `send()` faster than a slow client can drain, Node buffers it in memory. Check `ws.bufferedAmount` and drop or throttle rather than OOM the server.
- **Reaching for WebSockets when SSE would do.** If data only flows server-to-client (a live feed, a notification bell, LLM token streaming), SSE is simpler, survives proxies better, and reconnects for free.

## Best practices

- **Authenticate during the upgrade,** not after the socket opens — reject bad tokens before the connection exists, and re-check authorization on sensitive actions since a long-lived socket outlives a short-lived token.
- **Always run a heartbeat** (ping/pong on a timer) and terminate peers that miss it. Pair it with client-side reconnect-with-backoff.
- **Use `wss://` (TLS) in production.** Besides encryption, TLS dramatically improves the odds that intermediary proxies leave the upgraded connection alone.

- **Design idempotent, versioned messages.** On reconnect a client may replay or miss messages; a sequence number or "since" cursor lets you resync cleanly.
- **Plan for horizontal scale from day one** with a pub/sub backplane, even if you launch on one box — retrofitting it under load is painful.
- **Cap and account for connection count.** Each open socket is memory and a file descriptor; know your ceiling, set OS limits, and shed load gracefully.
- **Keep the protocol boundary clear:** use `ws` when you want a thin, standards-pure layer; use `socket.io` when you want rooms, acks, and fallbacks and control both ends.

## ⭐ Interview lens

**Q: Walk me through what happens when a browser opens a WebSocket.** The client sends an ordinary HTTP/1.1 GET with `Upgrade: websocket`, `Connection: Upgrade`, and a random `Sec-WebSocket-Key`. If the server accepts, it returns **101 Switching Protocols** with a `Sec-WebSocket-Accept` hash of that key. The same TCP connection then carries bidirectional WebSocket frames — no more HTTP request/response. In Node this surfaces as the `'upgrade'` event on the HTTP server.

**Q: WebSockets vs SSE vs polling — when do you pick each?** Polling for infrequent updates or dead-simple integrations. SSE when data flows *only* server-to-client (feeds, notifications, token streaming) — it is HTTP-native and auto-reconnects. WebSockets when you need genuinely *two-way*, low-latency traffic (chat, games, collaboration). *Trap:* candidates default to WebSockets for a live feed that has no client-to-server messages — that is SSE's job and it is cheaper to operate.

**Q: Your chat works locally but messages vanish after you scale to three instances. Why?** WebSocket connections are stateful and pinned to one instance. A broadcast on instance 1 never reaches clients held by instances 2 and 3. You

need a backplane — e.g. the socket.io Redis adapter or your own Redis pub/sub — so instances relay events to each other.

**Q: How do you detect a client that dropped without a proper close?** TCP may not notice for minutes. Send periodic pings; if a client misses N pongs, `terminate()` it. The classic implementation flags each socket `isAlive=false` before each ping and back to `true` on pong.

**Q: Is a WebSocket subject to the same-origin policy / CORS?** No — browsers do *not* apply CORS to WebSocket handshakes. That is exactly why Cross-Site WebSocket Hijacking exists. You must validate the `Origin` header and authenticate the connection yourself. *Trap:* assuming CORS "protects" the endpoint.

**Q: What is backpressure on a WebSocket and how do you handle it?** If you send faster than a client can receive, Node buffers unsent bytes in memory (`ws.bufferedAmount`). Under sustained pressure that is an OOM risk. Handle it by checking buffered amount and throttling, coalescing, or dropping stale updates for slow consumers.

## Cheat-card

- **WebSocket =** one long-lived, full-duplex TCP connection; starts as HTTP, upgrades via **101 Switching Protocols**.
- **Direction test:** two-way → WebSocket; server-only push → SSE; rare updates → polling.
- **Node hook:** `http.Server` `'upgrade'` event; `ws` = thin, `socket.io` = rooms + acks + fallback (not wire-compatible with `ws`).
- **Scale:** connections are pinned to one instance → need a Redis/pub-sub backplane; long-polling fallback needs sticky sessions.

- **Health:** ping/pong heartbeat + client reconnect-with-backoff; watch `bufferedAmount` for backpressure.
- **Security:** authenticate on upgrade, validate `Origin` (no CORS on WS), use `wss://`.
- **Frames:** tiny header, text/binary/ping/pong/close opcodes; browser-to-server frames are masked.

## QUICK CHECK — no marks, just for you.

1. How does a WebSocket connection begin?

- ◯ With a raw TCP handshake that never involves HTTP
- ◯ As an ordinary HTTP/1.1 GET with Upgrade: websocket headers; the server replies 101 Switching Protocols and the same TCP connection then carries frames
- ◯ With a POST to /graphql that returns a socket token
- ◯ By opening a second port dedicated to WebSocket traffic

Check answer

2. Your chat works locally but messages vanish after scaling to three instances. Why?

- ◯ WebSocket connections are stateful and pinned to one instance, so a broadcast on instance 1 never reaches clients held by instances 2 and 3; you need a Redis/pub-sub backplane
- ◯ The Origin header is being rejected by the load balancer
- ◯ TLS termination strips the frames between instances
- ◯ Each instance uses a different port so clients disconnect

Check answer

3. Are WebSocket connections subject to the browser's same-origin / CORS policy?

- ○ Yes, browsers block cross-origin WebSocket connections automatically
- ○ Yes, but only when using wss:// over TLS
- ○ No — browsers do not apply CORS to WebSocket handshakes, which is why you must validate the Origin header and authenticate on upgrade
- ○ No, because WebSockets can only ever connect to the same host

Check answer

‹ Previous
Graceful shutdown & monitoring

Next ›
GraphQL basics

## LEARN MORE

### ARTICLES & DOCS

- Socket.IO v4 Documentation - Introduction
  socket.io
- The WebSocket API (MDN)
  developer.mozilla.org

### VIDEOS




WebSockets in 100 Seconds & Beyond with Socket.io
Fireship

Hand-picked references for this lesson.